

Regularization (mathematics)

In mathematics, statistics, finance,^[1] and computer science, particularly in machine learning and inverse problems, **regularization** is a process that converts the answer to a problem to a simpler one. It is often used in solving ill-posed problems or to prevent overfitting.^[2]

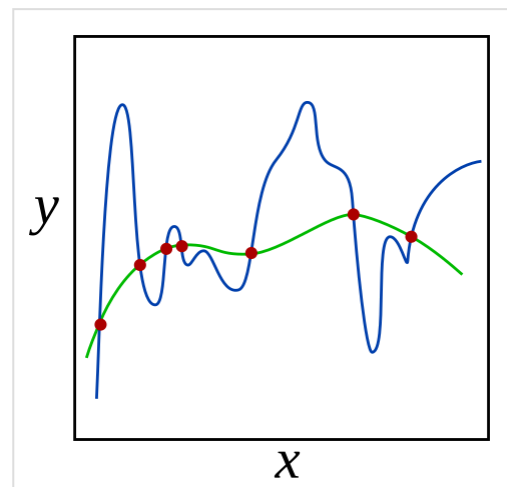
Although regularization procedures can be divided in many ways, the following delineation is particularly helpful:

- **Explicit regularization** is regularization whenever one explicitly adds a term to the optimization problem. These terms could be priors, penalties, or constraints. Explicit regularization is commonly employed with ill-posed optimization problems. The regularization term, or penalty, imposes a cost on the optimization function to make the optimal solution unique.
- **Implicit regularization** is all other forms of regularization. This includes, for example, early stopping, using a robust loss function, and discarding outliers. Implicit regularization is essentially ubiquitous in modern machine learning approaches, including stochastic gradient descent for training deep neural networks, and ensemble methods (such as random forests and gradient boosted trees).

In explicit regularization, independent of the problem or model, there is always a data term, that corresponds to a likelihood of the measurement, and a regularization term that corresponds to a prior. By combining both using Bayesian statistics, one can compute a posterior, that includes both information sources and therefore stabilizes the estimation process. By trading off both objectives, one chooses to be more aligned to the data or to enforce regularization (to prevent overfitting). There is a whole research branch dealing with all possible regularizations. In practice, one usually tries a specific regularization and then figures out the probability density that corresponds to that regularization to justify the choice. It can also be physically motivated by common sense or intuition.

In machine learning, the data term corresponds to the training data and the regularization is either the choice of the model or modifications to the algorithm. It is always intended to reduce the generalization error, i.e. the error score with the trained model on the evaluation set (testing data) and not the training data.^[3]

One of the earliest uses of regularization is Tikhonov regularization (ridge regression), related to the method of least squares.



The green and blue functions both incur zero loss on the given data points. A learned model can be induced to prefer the green function, which may generalize better to more points drawn from the underlying unknown distribution, by adjusting λ , the weight of the regularization term.

Regularization in machine learning

In machine learning, a key challenge is enabling models to accurately predict outcomes on unseen data, not just on familiar training data. Regularization is crucial for addressing overfitting—where a model memorizes training data details but cannot generalize to new data. The goal of regularization is to encourage models to learn the broader patterns within the data rather than memorizing it. Techniques like early stopping, L1 and L2 regularization, and dropout are designed to prevent overfitting and underfitting, thereby enhancing the model's ability to adapt to and perform well with new data, thus improving model generalization.^[4]

Early stopping

Stops training when validation performance deteriorates, preventing overfitting by halting before the model memorizes training data.^[4]

L1 and L2 regularization

Adds penalty terms to the cost function to discourage complex models:

- **L1 regularization** (also called LASSO) leads to sparse models by adding a penalty based on the absolute value of coefficients.
- **L2 regularization** (also called ridge regression) encourages smaller, more evenly distributed weights by adding a penalty based on the square of the coefficients.^[4]

Dropout

In the context of neural networks, the Dropout technique repeatedly ignores random subsets of neurons during training, which simulates the training of multiple neural network architectures at once to improve generalization.^[4]

Classification

Empirical learning of classifiers (from a finite data set) is always an underdetermined problem, because it attempts to infer a function of any $\mathbf{x}$ given only examples $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$.

A regularization term (or regularizer) $R(\mathbf{f})$ is added to a loss function:

$$\min_{\mathbf{f}} \sum_{i=1}^n V(\mathbf{f}(\mathbf{x}_i), y_i) + \lambda R(\mathbf{f})$$

where V is an underlying loss function that describes the cost of predicting $\mathbf{f}(\mathbf{x})$ when the label is $\mathbf{y}$, such as the square loss or hinge loss; and λ is a parameter which controls the importance of the regularization term. $R(\mathbf{f})$ is typically chosen to impose a penalty on the complexity of $\mathbf{f}$. Concrete notions of complexity used include restrictions for smoothness and bounds on the vector space norm.^[5]

A theoretical justification for regularization is that it attempts to impose Occam's razor on the solution (as depicted in the figure above, where the green function, the simpler one, may be preferred). From a Bayesian point of view, many regularization techniques correspond to imposing certain prior distributions on model parameters.^[6]

Regularization can serve multiple purposes, including learning simpler models, inducing models to be sparse and introducing group structure into the learning problem.

The same idea arose in many fields of science. A simple form of regularization applied to integral equations (Tikhonov regularization) is essentially a trade-off between fitting the data and reducing a norm of the solution. More recently, non-linear regularization methods, including total variation regularization, have become popular.

Generalization

Regularization can be motivated as a technique to improve the generalizability of a learned model.

The goal of this learning problem is to find a function that fits or predicts the outcome (label) that minimizes the expected error over all possible inputs and labels. The expected error of a function f_n is:

$$I[f_n] = \int_{X \times Y} V(f_n(x), y) \rho(x, y) dx dy$$

where X and Y are the domains of input data x and their labels y respectively.

Typically in learning problems, only a subset of input data and labels are available, measured with some noise. Therefore, the expected error is unmeasurable, and the best surrogate available is the empirical error over the N available samples:

$$I_S[f_n] = \frac{1}{n} \sum_{i=1}^N V(f_n(\hat{x}_i), \hat{y}_i)$$

Without bounds on the complexity of the function space (formally, the reproducing kernel Hilbert space) available, a model will be learned that incurs zero loss on the surrogate empirical error. If measurements (e.g. of x_i) were made with noise, this model may suffer from overfitting and display poor expected error. Regularization introduces a penalty for exploring certain regions of the function space used to build the model, which can improve generalization.

Tikhonov regularization (ridge regression)

These techniques are named for Andrey Nikolayevich Tikhonov, who applied regularization to integral equations and made important contributions in many other areas.

When learning a linear function f , characterized by an unknown vector w such that $f(x) = w \cdot x$, one can add the L_2 -norm of the vector w to the loss expression in order to prefer solutions with smaller norms. Tikhonov regularization is one of the most common forms. It is also known as ridge regression. It is expressed as:

$$\min_w \sum_{i=1}^n V(\hat{x}_i \cdot w, \hat{y}_i) + \lambda \|w\|_2^2,$$

where $(\hat{x}_i, \hat{y}_i)$, $1 \leq i \leq n$, would represent samples used for training.

In the case of a general function, the norm of the function in its reproducing kernel Hilbert space is:

$$\min_f \sum_{i=1}^n V(f(\hat{x}_i), \hat{y}_i) + \lambda \|f\|_{\mathcal{H}}^2$$

As the L_2 norm is differentiable, learning can be advanced by gradient descent.

Tikhonov-regularized least squares

The learning problem with the least squares loss function and Tikhonov regularization can be solved analytically. Written in matrix form, the optimal w is the one for which the gradient of the loss function with respect to w is 0.

$$\min_w \frac{1}{n} (\hat{X}w - Y)^T (\hat{X}w - Y) + \lambda \|w\|_2^2$$

$$\nabla_w = \frac{2}{n} \hat{X}^T (\hat{X}w - Y) + 2\lambda w$$

$$0 = \hat{X}^T (\hat{X}w - Y) + n\lambda w$$

$$w = (\hat{X}^T \hat{X} + \lambda n I)^{-1} (\hat{X}^T Y)$$

where the third statement is a first-order condition.

By construction of the optimization problem, other values of w give larger values for the loss function. This can be verified by examining the second derivative ∇_{ww} .

During training, this algorithm takes $O(d^3 + nd^2)$ time. The terms correspond to the matrix inversion and calculating $X^T X$, respectively. Testing takes $O(nd)$ time.

Early stopping

Early stopping can be viewed as regularization in time. Intuitively, a training procedure such as gradient descent tends to learn more and more complex functions with increasing iterations. By regularizing for time, model complexity can be controlled, improving generalization.

Early stopping is implemented using one data set for training, one statistically independent data set for validation and another for testing. The model is trained until performance on the validation set no longer improves and then applied to the test set.

Theoretical motivation in least squares

Consider the finite approximation of Neumann series for an invertible matrix A where $\|I - A\| < 1$:

$$\sum_{i=0}^{T-1} (I - A)^i \approx A^{-1}$$

This can be used to approximate the analytical solution of unregularized least squares, if γ is introduced to ensure the norm is less than one.

$$w_T = \frac{\gamma}{n} \sum_{i=0}^{T-1} \left(I - \frac{\gamma}{n} \hat{X}^\top \hat{X} \right)^i \hat{X}^\top \hat{Y}$$

The exact solution to the unregularized least squares learning problem minimizes the empirical error, but may fail. By limiting T , the only free parameter in the algorithm above, the problem is regularized for time, which may improve its generalization.

The algorithm above is equivalent to restricting the number of gradient descent iterations for the empirical risk

$$I_s[w] = \frac{1}{2n} \|\hat{X}w - \hat{Y}\|_{\mathbb{R}^n}^2$$

with the gradient descent update:

$$w_0 = 0$$

$$w_{t+1} = \left(I - \frac{\gamma}{n} \hat{X}^\top \hat{X} \right) w_t + \frac{\gamma}{n} \hat{X}^\top \hat{Y}$$

The base case is trivial. The inductive case is proved as follows:

$$\begin{aligned} w_T &= \left(I - \frac{\gamma}{n} \hat{X}^\top \hat{X} \right) \frac{\gamma}{n} \sum_{i=0}^{T-2} \left(I - \frac{\gamma}{n} \hat{X}^\top \hat{X} \right)^i \hat{X}^\top \hat{Y} + \frac{\gamma}{n} \hat{X}^\top \hat{Y} \\ &= \frac{\gamma}{n} \sum_{i=1}^{T-1} \left(I - \frac{\gamma}{n} \hat{X}^\top \hat{X} \right)^i \hat{X}^\top \hat{Y} + \frac{\gamma}{n} \hat{X}^\top \hat{Y} \\ &= \frac{\gamma}{n} \sum_{i=0}^{T-1} \left(I - \frac{\gamma}{n} \hat{X}^\top \hat{X} \right)^i \hat{X}^\top \hat{Y} \end{aligned}$$

Regularizers for sparsity

Assume that a dictionary ϕ_j with dimension p is given such that a function in the function space can be expressed as:

$$f(x) = \sum_{j=1}^p \phi_j(x) w_j$$

Enforcing a sparsity constraint on w can lead to simpler and more interpretable models. This is useful in many real-life applications such as computational biology. An example is developing a simple predictive test for a disease in order to minimize the cost of performing medical tests while

maximizing predictive power.

A sensible sparsity constraint is the L_0 norm $\|w\|_0$, defined as the number of non-zero elements in w . Solving a L_0 regularized learning problem, however, has been demonstrated to be NP-hard.^[7]

The L_1 norm (see also Norms) can be used to approximate the optimal L_0 norm via convex relaxation. It can be shown that the L_1 norm induces sparsity. In the case of least squares, this problem is known as LASSO in statistics and basis pursuit in signal processing.

$$\min_{w \in \mathbb{R}^p} \frac{1}{n} \|\hat{X}w - \hat{Y}\|^2 + \lambda \|w\|_1$$

L_1 regularization can occasionally produce non-unique solutions. A simple example is provided in the figure when the space of possible solutions lies on a 45 degree line. This can be problematic for certain applications, and is overcome by combining L_1 with L_2 regularization in elastic net regularization, which takes the following form:

$$\min_{w \in \mathbb{R}^p} \frac{1}{n} \|\hat{X}w - \hat{Y}\|^2 + \lambda \left(\alpha \|w\|_1 + (1 - \alpha) \|w\|_2^2 \right), \alpha$$

Elastic net regularization tends to have a grouping effect, where correlated input features are assigned equal weights.

Elastic net regularization is commonly used in practice and is implemented in many machine learning libraries.

Proximal methods

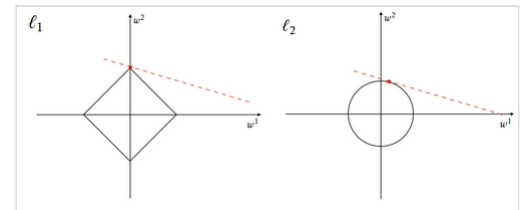
While the L_1 norm does not result in an NP-hard problem, the L_1 norm is convex but is not strictly differentiable due to the kink at $x = 0$. Subgradient methods which rely on the subderivative can be used to solve L_1 regularized learning problems. However, faster convergence can be achieved through proximal methods.

For a problem $\min_{w \in H} F(w) + R(w)$ such that F is convex, continuous, differentiable, with Lipschitz continuous gradient (such as the least squares loss function), and R is convex, continuous, and proper, then the proximal method to solve the problem is as follows. First define the proximal operator

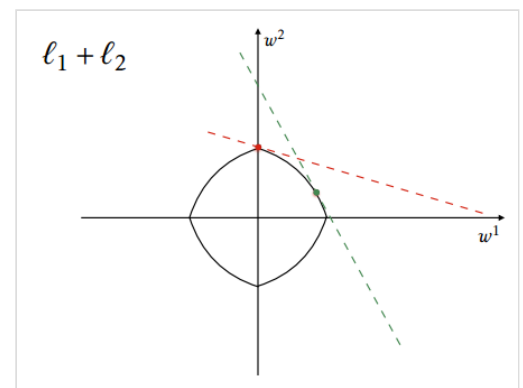
$$\text{prox}_R(v) = \underset{w \in \mathbb{R}^D}{\text{argmin}} \left\{ R(w) + \frac{1}{2} \|w - v\|^2 \right\},$$

and then iterate

$$w_{k+1} = \underset{\gamma, R}{\text{prox}}(w_k - \gamma \nabla F(w_k))$$



A comparison between the L1 ball and the L2 ball in two dimensions gives an intuition on how L1 regularization achieves sparsity.



Elastic net regularization

The proximal method iteratively performs gradient descent and then projects the result back into the space permitted by $\mathbf{R}$.

When $\mathbf{R}$ is the L_1 regularizer, the proximal operator is equivalent to the soft-thresholding operator,

$$S_\lambda(v)f(n) = \begin{cases} v_i - \lambda, & \text{if } v_i > \lambda \\ 0, & \text{if } v_i \in [-\lambda, \lambda] \\ v_i + \lambda, & \text{if } v_i < -\lambda \end{cases}$$

This allows for efficient computation.

Group sparsity without overlaps

Groups of features can be regularized by a sparsity constraint, which can be useful for expressing certain prior knowledge into an optimization problem.

In the case of a linear model with non-overlapping known groups, a regularizer can be defined:

$$R(w) = \sum_{g=1}^G \|w_g\|_2,$$

where

$$\|w_g\|_2 = \sqrt{\sum_{j=1}^{|G_g|} (w_g^j)^2}$$

This can be viewed as inducing a regularizer over the L_2 norm over members of each group followed by an L_1 norm over groups.

This can be solved by the proximal method, where the proximal operator is a block-wise soft-thresholding function:

$$\text{prox}_{\lambda, R, g}(w_g) = \begin{cases} \left(1 - \frac{\lambda}{\|w_g\|_2}\right) w_g, & \text{if } \|w_g\|_2 > \lambda \\ 0, & \text{if } \|w_g\|_2 \leq \lambda \end{cases}$$

Group sparsity with overlaps

The algorithm described for group sparsity without overlaps can be applied to the case where groups do overlap, in certain situations. This will likely result in some groups with all zero elements, and other groups with some non-zero and some zero elements.

If it is desired to preserve the group structure, a new regularizer can be defined:

$$R(w) = \inf \left\{ \sum_{g=1}^G \|w_g\|_2 : w = \sum_{g=1}^G \bar{w}_g \right\}$$

For each w_g , $\bar{w}_g$ is defined as the vector such that the restriction of $\bar{w}_g$ to the group g equals w_g and all other entries of $\bar{w}_g$ are zero. The regularizer finds the optimal disintegration of w into parts. It can be viewed as duplicating all elements that exist in multiple groups. Learning problems with this regularizer can also be solved with the proximal method with a complication. The proximal operator cannot be computed in closed form, but can be effectively solved iteratively, inducing an inner iteration within the proximal method iteration.

Regularizers for semi-supervised learning

When labels are more expensive to gather than input examples, semi-supervised learning can be useful. Regularizers have been designed to guide learning algorithms to learn models that respect the structure of unsupervised training samples. If a symmetric weight matrix W is given, a regularizer can be defined:

$$R(f) = \sum_{i,j} w_{ij} (f(x_i) - f(x_j))^2$$

If W_{ij} encodes the result of some distance metric for points x_i and x_j , it is desirable that $f(x_i) \approx f(x_j)$. This regularizer captures this intuition, and is equivalent to:

$$R(f) = \bar{f}^\top L \bar{f}$$

where $L = D - W$ is the Laplacian matrix of the graph induced by W .

The optimization problem $\min_{f \in \mathbb{R}^m} R(f)$, $m = u + l$ can be solved analytically if the constraint $f(x_i) = y_i$ is applied for all supervised samples. The labeled part of the vector f is therefore obvious. The unlabeled part of f is solved for by:

$$\min_{f_u \in \mathbb{R}^u} f^\top L f = \min_{f_u \in \mathbb{R}^u} \{ f_u^\top L_{uu} f_u + f_l^\top L_{lu} f_u + f_u^\top L_{ul} f_l \}$$

$$\nabla_{f_u} = 2L_{uu} f_u + 2L_{ul} Y$$

$$f_u = L_{uu}^\dagger (L_{ul} Y)$$

The pseudo-inverse can be taken because L_{ul} has the same range as L_{uu} .

Regularizers for multitask learning

In the case of multitask learning, T problems are considered simultaneously, each related in some way. The goal is to learn T functions, ideally borrowing strength from the relatedness of tasks, that have predictive power. This is equivalent to learning the matrix $W : T \times D$.

Sparse regularizer on columns

$$R(w) = \sum_{i=1}^D \|W\|_{2,1}$$

This regularizer defines an L2 norm on each column and an L1 norm over all columns. It can be solved by proximal methods.

Nuclear norm regularization

$$R(w) = \|\sigma(W)\|_1$$

where $\sigma(W)$ is the eigenvalues in the singular value decomposition of W .

Mean-constrained regularization

$$R(f_1 \cdots f_T) = \sum_{t=1}^T \left\| f_t - \frac{1}{T} \sum_{s=1}^T f_s \right\|_{H_k}^2$$

This regularizer constrains the functions learned for each task to be similar to the overall average of the functions across all tasks. This is useful for expressing prior information that each task is expected to share with each other task. An example is predicting blood iron levels measured at different times of the day, where each task represents an individual.

Clustered mean-constrained regularization

$$R(f_1 \cdots f_T) = \sum_{r=1}^C \sum_{t \in I(r)} \left\| f_t - \frac{1}{I(r)} \sum_{s \in I(r)} f_s \right\|_{H_k}^2$$

where $I(r)$ is a cluster of tasks.

This regularizer is similar to the mean-constrained regularizer, but instead enforces similarity between tasks within the same cluster. This can capture more complex prior information. This technique has been used to predict Netflix recommendations. A cluster would correspond to a group of people who share similar preferences.

Graph-based similarity

More generally than above, similarity between tasks can be defined by a function. The regularizer encourages the model to learn similar functions for similar tasks.

$$R(f_1 \cdots f_T) = \sum_{t,s=1, t \neq s}^T \|f_t - f_s\|^2 M_{ts}$$

for a given symmetric similarity matrix M .